

Stage 3 — System Design & Architecture

Stateful Application-Layer Firewall & Intrusion Filter

Comprehensive System Programming & C++ Project Documentation

Author: Ayush Behera

Abstract

This document describes the system design and architecture of the Stateful Application-Layer Firewall & Intrusion Filter. Stage 3 moves the project from the requirements defined in the PRD to a more detailed technical design that can be used during implementation.

It covers the overall system architecture, interaction between the Linux kernel and user space, major software components, core data structures, concurrency and synchronization methods, error handling, and the Git workflow that will be used during development. It also defines the planned UML diagrams and component interactions that will guide the implementation in the later stages.

1. Introduction & Stage 3 Engineering Objectives

As network systems become more complex, traditional perimeter-based security is often not enough to identify threats hidden inside legitimate network traffic. Stage 3 converts the requirements from the previous stage into a concrete software structure that can be implemented in C++.

The main objectives of this stage are:

- Establish a modular architecture that separates packet interception, connection tracking, protocol inspection, logging, and administrative control.
- Define the responsibilities and interfaces of the major components and describe how information moves between the user-space daemon and Linux networking components.
- Design suitable synchronization mechanisms so that the system can handle network traffic efficiently without introducing race conditions or unnecessary delays.
- Define the Class, Sequence, and State Machine models that will be used as references during the implementation stages.

2. Architectural Design Principles

The architecture of the firewall follows several important system-programming principles.

1. Separation of Concerns

Each subsystem should have a clearly defined responsibility. For example, the packet interceptor should handle communication with the Netfilter queue without knowing how application-layer signatures are evaluated. Similarly, the inspection engine should be independent of the low-level kernel queue and socket management.

2. Low Overhead

The packet-processing path should avoid unnecessary memory allocations and data copying wherever practical. This is important because every additional processing step can affect packet latency and overall throughput.

3. Exception Safety & RAII

Modern C++ resource-management techniques, particularly RAII and smart pointers, should be used to manage resources safely. This includes resources such as dynamically allocated memory, file descriptors, and other system handles.

4. Concurrency & Lock Granularity

The connection table may be accessed by multiple threads, so synchronization needs to be carefully designed. A `std::shared_mutex` can be used to allow multiple read operations while ensuring that state updates remain protected.

5. Runtime Configuration

Security rules and configuration should be updateable while the daemon is running. The design should allow rule changes without requiring the entire firewall process to restart or unnecessarily interrupt active connections.

3. High-Level System Architecture & Kernel-User Boundary

The firewall runs primarily as a privileged user-space daemon on Linux and communicates with the kernel through the Netfilter framework.

Packet Processing Pipeline

The expected packet-processing flow is:

- **Network Interface:** Network packets enter through the Linux network interface and are processed by the kernel's networking stack, including IP processing and routing.
- **Netfilter Queue:** Firewall rules configured using `iptables` or `nftables` can redirect selected packets to a Netfilter queue using `NF_QUEUE`. This allows the user-space firewall to inspect the packets before a final decision is made.
- **User-Space Polling:** The Interceptor module uses `libnetfilter_queue` to connect to the queue and receive packet information through the associated netlink socket.
- **Connection Tracking:** The intercepted packet is parsed into a normalized 5-tuple containing the source IP, destination IP, source port, destination port, and protocol. The `ConntrackTable` then checks whether the packet belongs to a known flow and validates its current state.
- **Layer 7 Inspection:** If the packet belongs to an appropriate connection and contains unencrypted application data, the payload is passed to the Inspector. For example, HTTP requests and DNS queries can be checked against configured signatures and rules.
- **Kernel Verdict:** After processing, the firewall sends a verdict back to the kernel using `nfq_set_verdict()`. Depending on the result, the packet can be accepted or dropped.
- **Logging:** Important events such as dropped packets, invalid state transitions, and signature matches are passed to the Logger for recording.

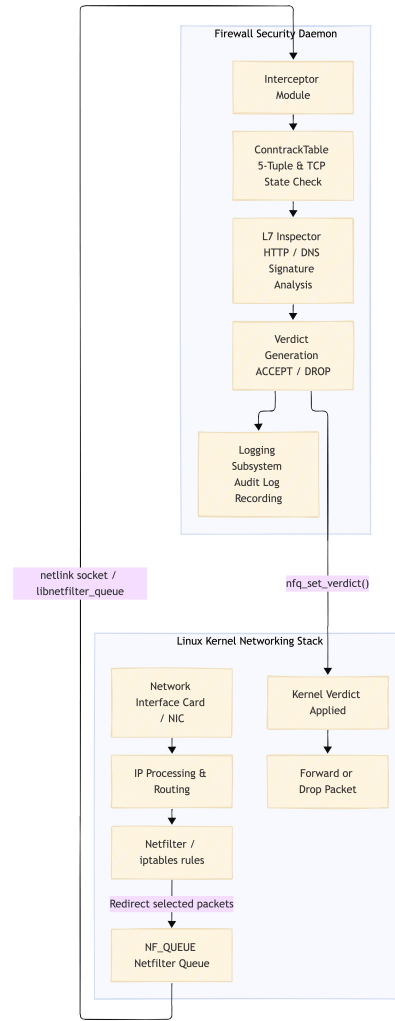


Figure 1: High-Level System Architecture and Packet Processing Pipeline. Shows the path of a packet from the Linux network interface through the kernel and Netfilter queue, into the user-space firewall (Interceptor, ConntrackTable, and Inspector), and finally back to the kernel with an accept or drop verdict[cite: 4].

4. Detailed Subsystem Specifications

4.1 Interceptor Subsystem (Interceptor)

- **Role:** Acts as the main interface between the Linux networking stack and the user-space firewall daemon.
- **Mechanism:** Initializes a Netfilter queue using `libnetfilter_queue`, registers a packet callback such as `nfq_callback`, and manages the associated netlink socket.
- **Responsibilities:** Extracts information such as the packet ID, payload length, and packet data from the Netfilter callback. This information is then placed into a lightweight C++ packet structure and passed to the other processing modules. Once processing is complete, the interceptor sends the appropriate verdict using `nfq_set_verdict()`.

4.2 Stateful Connection Tracker (ConntrackTable)

- **Role:** Keeps track of active network connections and provides context when deciding whether packets belong to valid flows.

- **Mechanism:** Uses an in-memory hash table where each connection is identified by a normalized 5-tuple consisting of the source IP, destination IP, source port, destination port, and protocol.
- **State Machine:** Tracks TCP control flags such as SYN, ACK, FIN, and RST and compares them with the current connection state, including states such as `SYN_SENT`, `ESTABLISHED`, `FIN_WAIT`, and `CLOSED`.
- **Garbage Collection:** A background thread periodically checks the connection table and removes stale, half-open, or timed-out connections. This prevents unused entries from accumulating and consuming excessive memory.

4.3 Layer 7 Inspection Engine (Inspector)

- **Role:** Examines application-layer data for suspicious patterns, protocol anomalies, and possible injection attacks.
- **Mechanism:** Initially focuses on unencrypted application data such as HTTP request methods, URIs, headers, and basic DNS query packets.
- **Pattern Matching:** Extracted application data is compared against configured signatures and regular-expression patterns. The initial rule set can include patterns associated with attacks such as SQL injection (SQLi) and Cross-Site Scripting (XSS).

4.4 Management CLI & IPC Subsystem (fw_admin)

- **Role:** Provides an administrative interface for interacting with the running firewall daemon.
- **Mechanism:** The CLI communicates with the daemon using a local IPC mechanism such as pipes or UNIX domain sockets.
- **Capabilities:** Administrators can use the CLI to reload rule files, view current connection statistics, change logging levels, and check packet-drop statistics without restarting the daemon.

4.5 Logging & Auditing Subsystem (Logger)

- **Role:** Records system activity and security-related events for monitoring and later analysis.
- **Mechanism:** Provides thread-safe asynchronous logging with different severity levels, including `DEBUG`, `INFO`, `WARNING`, `ERROR`, and `CRITICAL`.
- **Formatting:** Log entries include timestamps with microsecond-level precision and are written to persistent storage. Log rotation can also be used to prevent log files from growing indefinitely.

5. UML Design Specifications & Component Modeling

5.1 Class Diagram Architecture

The software is organized around several main C++ classes, with each class handling a specific part of the firewall.

- **FirewallDaemon:** Acts as the main controller of the application. It creates and manages the `Interceptor`, `ConntrackTable`, `Inspector`, and `Logger` components. It also handles signals such as `SIGINT` and `SIGTERM` so that the daemon can shut down cleanly.
- **Interceptor:** Manages the Netfilter queue, socket descriptors, and packet callbacks. It provides methods such as `initQueue()`, `registerCallback()`, and `setVerdict()`.

- **ConntrackTable:** Manages the connection hash table and the synchronization mechanisms used to protect it. Important operations include `lookup()`, `updateState()`, and `gcCleanup()`.
- **Inspector:** Contains the application-layer parsing and signature-matching logic. Example methods include `parseHttp()`, `matchSignatures()`, and `loadRules()`.
- **Logger:** Handles the formatting, queuing, and storage of security and system log messages.

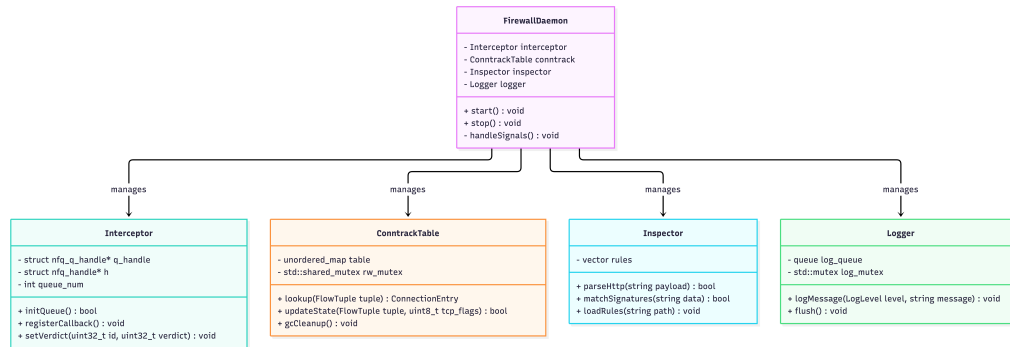


Figure 2: System Class Diagram (**FirewallDaemon**, **Interceptor**, **ConntrackTable**, **Inspector**, **Logger**). Shows the main C++ classes, their important attributes and methods, and the relationships between the central daemon and its supporting modules[cite: 4].

5.2 Sequence Flow Specifications — Packet Lifecycle

The normal packet-processing sequence is expected to follow these steps:

1. **Kernel Interception:** The Linux kernel processes a network packet and, based on the configured firewall rules, sends it to `NF_QUEUE`.
2. **Queue Notification:** `libnetfilter_queue` receives the queued packet information through the netlink socket.
3. **Metadata Extraction:** The Interceptor extracts the packet metadata and creates the internal packet representation.
4. **State Validation:** The packet is passed to the **ConntrackTable**, which performs the 5-tuple lookup and checks the current TCP state.
5. **Payload Inspection:** If the packet passes the state checks and contains relevant application data, it is passed to the Layer 7 Inspector.
6. **Verdict Execution:** The inspection result is returned to the Interceptor, which calls `nfq_set_verdict()` to either accept or drop the packet.

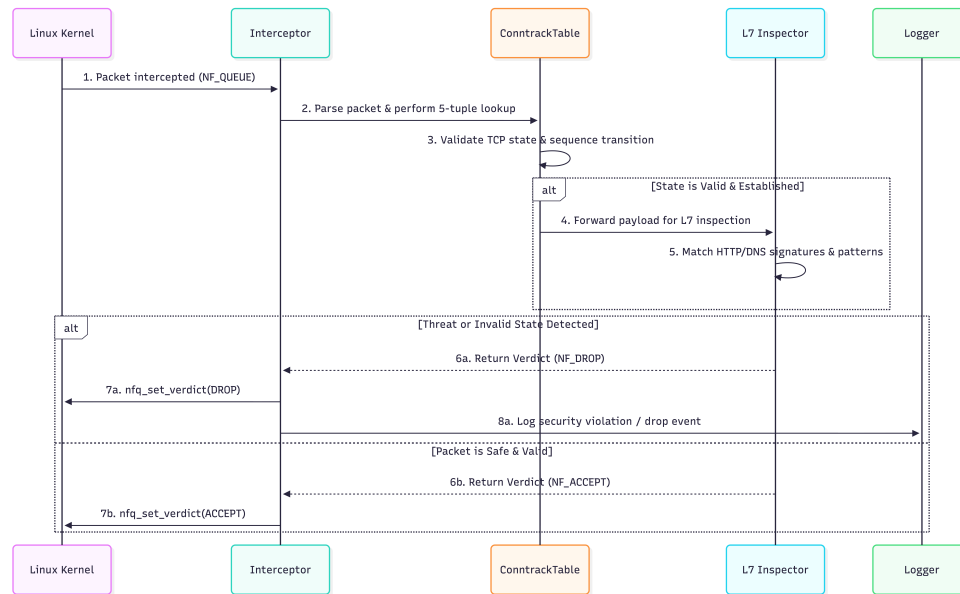


Figure 3: Sequence Flow Diagram — Kernel Packet Interception and Verdict Lifecycle. Shows the order of communication between the Linux kernel, Interceptor, ConntrackTable, and Inspector as a packet moves through the firewall[cite: 4].

5.3 State Machine Transition Specifications

The connection tracker will use a simplified TCP state machine for validating connection progress.

- **CLOSED / NONE:** The default state when no active connection exists. When an appropriate TCP SYN packet is received, a new connection entry is created and moves into the handshake state.
- **SYN_SENT:** Represents the initial connection-establishment phase. When the expected response, such as a valid SYN-ACK, is received, the connection can progress toward the established state.
- **ESTABLISHED:** Represents the normal data-transfer phase. Packets in this state can undergo both connection-state validation and Layer 7 inspection. Receiving a FIN or RST can move the connection toward termination.

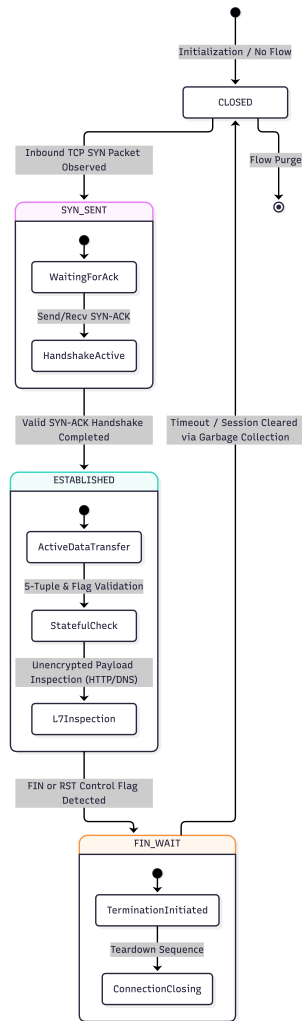


Figure 4: TCP State Machine Transition Diagram. Shows the main TCP connection states, including CLOSED, SYN_SENT, ESTABLISHED, and FIN_WAIT, along with the control flags that trigger state transitions[cite: 4].

6. Core Data Structures & API Interface Design

The system uses strongly typed and efficient C++ data structures to keep packet processing predictable and manageable.

- **Protocol Enum:** A strongly typed enumeration using `uint8_t` to represent transport protocols such as TCP = 6, UDP = 17, and ICMP = 1.
- **TcpState Enum:** Represents the different stages of a TCP connection, including NONE, SYN_SENT, SYN_RECV, ESTABLISHED, FIN_WAIT, CLOSE_WAIT, and CLOSED.
- **FlowTuple Struct:** Stores the source IP, destination IP, source port, destination port, and protocol as a single flow identifier. It also provides the equality operations needed for hash-table lookups.
- **ConnectionEntry Struct:** Stores a FlowTuple together with the current TCP state, packet and byte counters, and a `std::chrono::steady_clock::time_point` value used to determine when a connection has timed out.

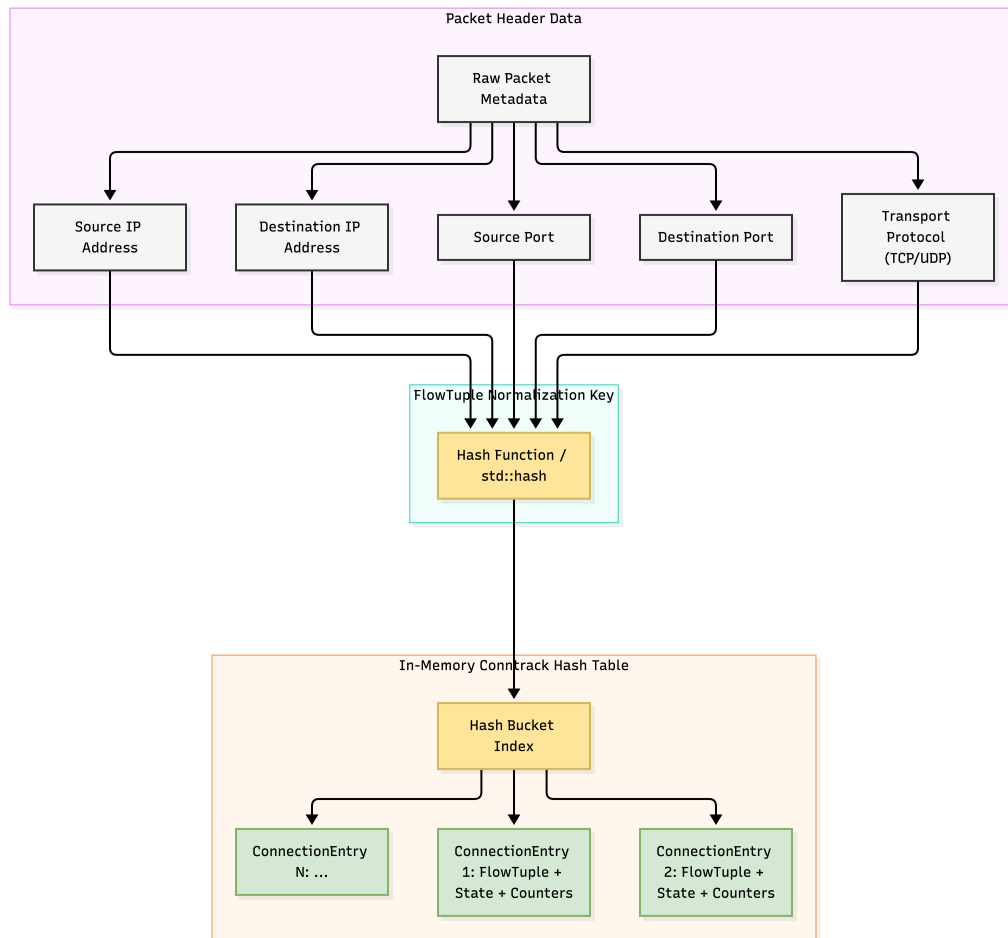


Figure 5: Core Data Structures and Flow Tuple Hash Map Layout. Shows how a 5-tuple is converted into a hash key and mapped to the corresponding ConnectionEntry stored in the connection table[cite: 4].

7. Concurrency, Synchronization & Thread Safety

Network packets can arrive continuously and sometimes in large bursts, so the system needs to handle concurrent operations safely.

- **Reader-Writer Locking:** The `ConntrackTable` can use `std::shared_mutex` to allow multiple threads to perform read-only lookups at the same time. Operations that modify or insert connection entries use an exclusive lock through `std::unique_lock`.
- **Asynchronous Logging:** The Logger can use a producer-consumer queue protected by a mutex or another suitable synchronization mechanism. Packet-processing threads can place log messages into the queue without waiting for disk I/O to complete.
- **Garbage Collection Thread:** A separate background thread periodically scans the connection table and removes expired entries. It acquires the necessary exclusive lock only during the parts of the cleanup operation that modify the shared table.

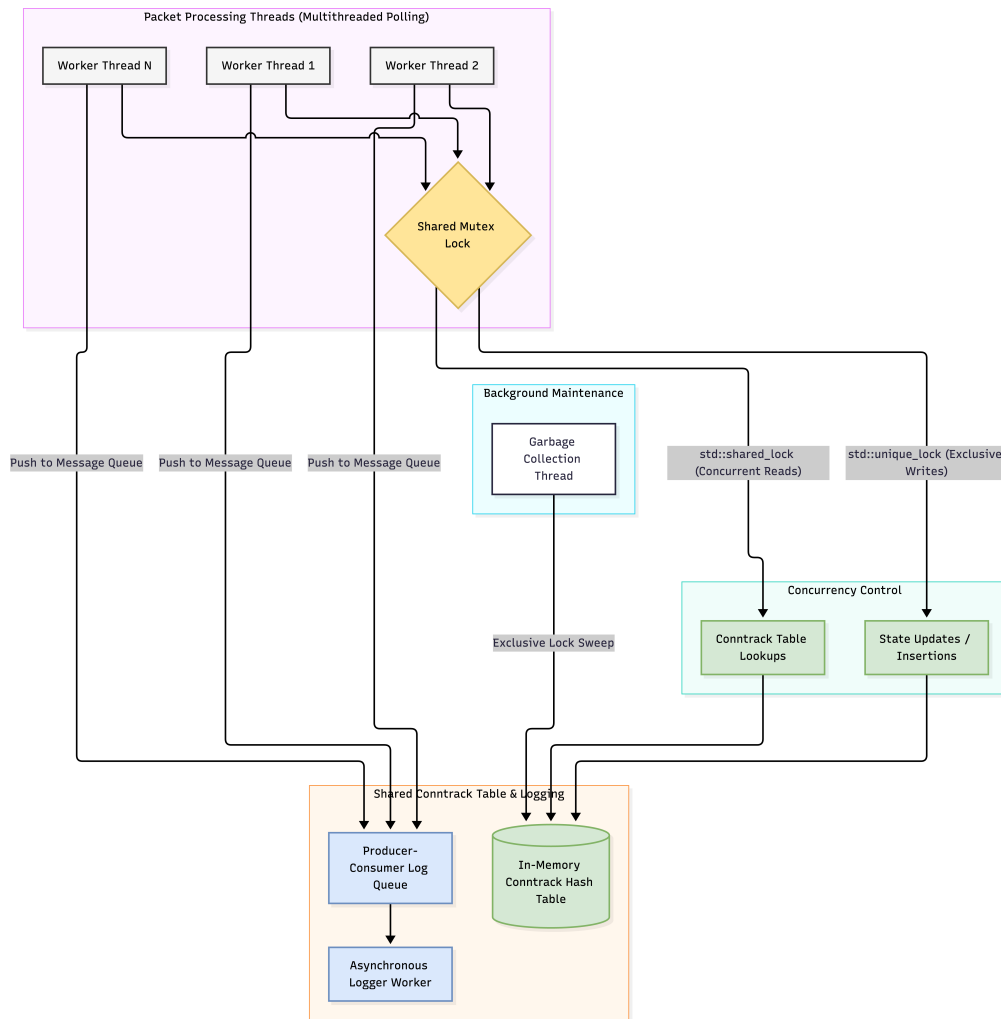


Figure 6: Concurrency Model and Shared Mutex Reader-Writer Architecture. Illustrates the worker threads accessing the connection table, shared locks used for lookups, exclusive locks used for updates, asynchronous logging, and the background cleanup thread[cite: 4].

8. Error Handling, Fault Tolerance & Graceful Shutdown

Since the firewall operates close to the Linux networking stack, it needs to handle failures carefully and release system resources correctly.

- **Socket Recovery:** If communication with the Netfilter queue is temporarily interrupted, the Interceptor should attempt to recover the connection where possible rather than immediately terminating the entire daemon.
- **Resource Cleanup:** File descriptors, Netfilter queue handles, and dynamically allocated resources should be managed using RAII-based wrappers so that they are released correctly during normal shutdown and error conditions.
- **Signal Handling:** The daemon should handle `SIGINT` and `SIGTERM` to support a controlled shutdown. During shutdown, it should flush pending log messages, release Netfilter resources, clean up firewall-related configuration where required, and exit without leaving unnecessary resources behind.

9. Git Repository Setup & Branching Strategy

To keep development organized, the project will follow a GitFlow-style branching structure.

- **main:** Contains stable versions of the project that are ready for demonstration or release.
- **develop:** Acts as the main integration branch where completed features from individual development branches are combined and tested.
- **feature/stage3-architecture:** Used specifically for Stage 3 work, including architecture documentation, UML diagrams, and initial structural definitions.

Commit Conventions

Commits should use short and descriptive prefixes to make the project history easier to understand. Examples include:

- **feat:** `implement conntrack table structure`
- **docs:** `add stage 3 architecture specification`
- **fix:** `resolve shared_mutex deadlock`

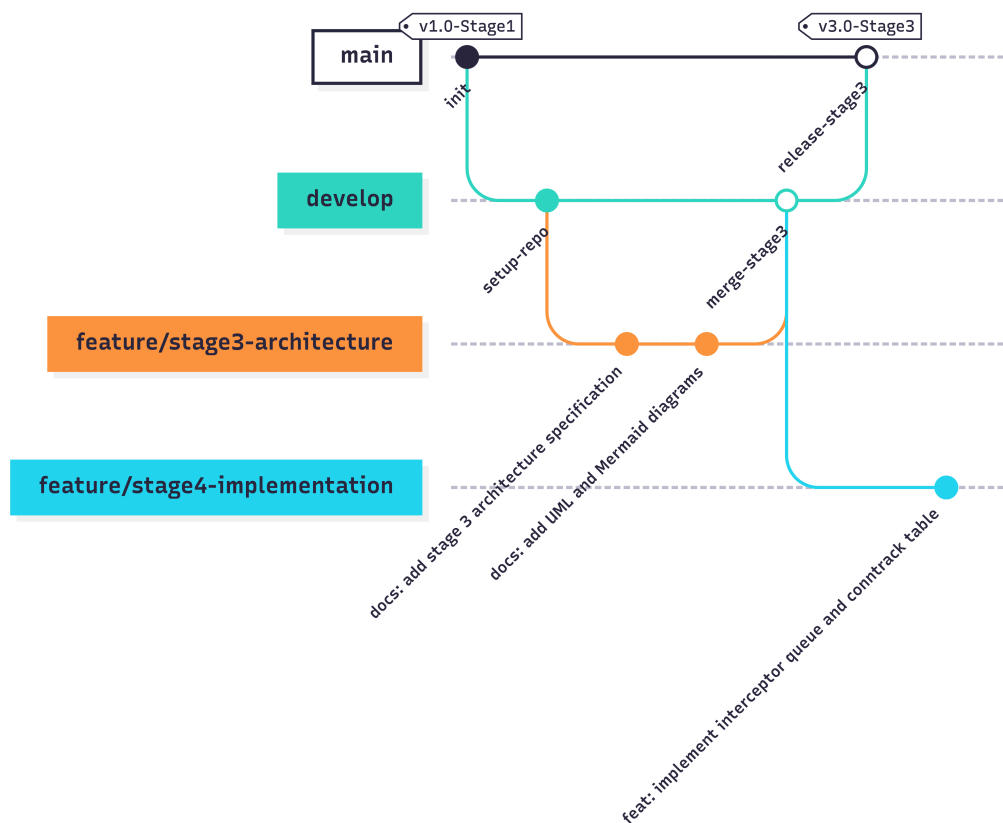


Figure 7: GitFlow Branching Strategy Diagram. Shows the relationship between main, develop, and feature branches, including the process of integrating changes and creating stable releases[cite: 4].

10. Conclusion & Roadmap to Stage 4

Stage 3 defines the main architecture of the Stateful Application-Layer Firewall & Intrusion Filter, including its major components, data structures, packet-processing flow, concurrency model, error-handling approach, and development workflow.

The architecture provides a clear structure for implementing the system in C++ while keeping

packet interception, connection tracking, application-layer inspection, logging, and administration as separate components.

With the design now defined, the next stage is **Stage 4 — Initial Implementation & Prototype**. The focus will move from architecture to actual C++ development, beginning with the core files such as `Interceptor.h/.cpp` and `ConntrackTable.h/.cpp`, followed by compilation, testing, and the first working prototype.